QueueList Reflection Log
Noorinder

Credit Name: Chapter 13
Assignment Name: QueueList

How has your program changed from planning to coding to now? Please explain.

Initially, the goal was to create a simple queue implementation using a linked list structure. The plan included building a `QueueList` class with core methods for adding, removing, and inspecting elements. While the overall structure followed the plan, unexpected issues during coding required modifications and improvements to ensure functionality and robustness.

1. Incorrect Front Assignment During Enqueue
Problem:
In the `enqueue()` method, the `front` pointer was not properly set when the queue was empty, leading to an invalid state.

Fix:
I updated the code to assign the new node to both `front` and `rear` when adding the first element.

Code Example:

```
public void enqueue(Object item) {
    Node newNode = new Node(item);
    if (isEmpty()) { // When the queue is empty
        front = newNode; // Set front to the new node
        rear = newNode;  // Also set rear to the new node
    } else {
        rear.setNext(newNode); // Link the new node to the existing rear
        rear = newNode;        // Update rear to the new node
    }
}
```

2. NullPointerException on Dequeue from Empty Queue
Problem:
The `dequeue()` method did not handle scenarios where the queue was already empty, causing runtime errors.

Fix:
I added a condition to check if the queue was empty and returned an appropriate message instead of proceeding with invalid operations.

Code Example:

```
public Object dequeue() {
    if (isEmpty()) { // Check if the queue is empty
        System.out.println("Queue is empty. Cannot dequeue."); // Return an error message
        return null;
    }
    Object data = front.getData(); // Get the front item
    front = front.getNext(); // Move front to the next node
    if (front == null) { // If the queue is now empty
        rear = null; // Update rear to null as well
    }
    return data; // Return the dequeued item
}
```

3. Unlinked Nodes During Enqueue
Problem:
When adding a new element, the existing `rear` node was not connected to the new node, causing a break in the queue's continuity.

Fix:
I added logic to set the `rear` node's `next` reference to the new node before updating the `rear` pointer.

Code Example:

```
public void enqueue(Object item) {
    Node newNode = new Node(item);
    if (isEmpty()) {
        front = newNode;
        rear = newNode;
    } else {
        rear.setNext(newNode); // Link the new node to the current rear
        rear = newNode;        // Update the rear reference
    }
}
```

4. Size Calculation Inefficiency
Problem:
The `size()` method traversed the entire queue every time, which was inefficient for larger queues.

Fix:
I introduced a counter variable that increments or decrements during `enqueue()` and `dequeue()` operations to track the size directly.

Code Example:

```
private int size; // Add a size variable to track the number of elements

public QueueList() {
    front = null;
    rear = null;
    size = 0; // Initialize size to 0
}

public void enqueue(Object item) {
    Node newNode = new Node(item);
    if (isEmpty()) {
        front = newNode;
        rear = newNode;
    } else {
        rear.setNext(newNode);
        rear = newNode;
    }
    size++; // Increment size when adding an element
}

public Object dequeue() {
    if (isEmpty()) {
        System.out.println("Queue is empty. Cannot dequeue.");
        return null;
    }
    Object data = front.getData();
    front = front.getNext();
    if (front == null) {
        rear = null;
    }
    size--; // Decrement size when removing an element
    return data;
}

public int size() {
    return size; // Return the size variable directly
}
```

5. Improper Error Handling in Peek
Problem:
The `peek()` method attempted to access data even when the queue was empty, resulting in runtime exceptions.

Fix:
I added a check to return a message indicating that the queue is empty instead of accessing invalid data.

Code Example:

```
public Object peek() {
    if (isEmpty()) { // Check if the queue is empty
        System.out.println("Queue is empty. No front item to peek."); // Error message
        return null;
    }
    return front.getData(); // Return the data at the front
}
```

Final Thoughts:

These changes significantly improved the queue's stability and usability. By addressing edge cases and enhancing error handling, the program now manages operations like enqueue, dequeue, and size retrieval smoothly. The modifications not only solved the immediate problems but also made the code more efficient and user-friendly.